

Практическая работа №43

Выполнил:
Губин А. С
Проверила:
Зиберева А. А

Ход выполнения работы

1. Подготовка и инициализация проекта

Работу я начал с настройки рабочего окружения в терминале VS Code.

- Сначала инициализировал проект командой `npm init -y`. Это создало файл `package.json`, который необходим для управления зависимостями.
- Затем я создал файл `.gitignore` командой `npm gitignore node`. Это важное действие, чтобы при загрузке на GitHub в репозиторий не попали лишние системные файлы и тяжелая папка `node_modules`.

2. Установка и настройка TypeScript

Для превращения кода из TS в понятный браузеру или серверу JS, я установил необходимые инструменты:

- Выполнил команду `npm i -D typescript @types/node`. Установил их как зависимости разработки (-D), так как они нужны только на этапе написания кода.
- Далее сгенерировал файл конфигурации командой `npm tsc --init`. В настройках `tsconfig.json` я указал, что исходные файлы будут лежать в папке `src`, а скомпилированные — в папке `dist`. Также включил строгий режим (`strict`), чтобы TypeScript указывал мне на малейшие неточности в типах.

3. Написание кода (src/pract.ts)

Я создал основной файл `src/pract.ts` и реализовал в нем примеры, демонстрирующие ключевые возможности языка:

- **Аннотации типов:** Жестко закрепил типы для базовых переменных (числа, строки, массивы).
- **Интерфейсы (User):** Описал структуру объекта пользователя с обязательными и опциональными полями.
- **Специальные типы:** Использовал `Union` (когда ID может быть и числом, и строкой) и `Intersection` (для склейки свойств разных интерфейсов).
- **Продвинутые инструменты:** Создал перечисление `Enum` для ошибок входа, реализовал универсальное хранилище данных через `Generics` и защитил данные от изменений с помощью `Readonly`.

4. Настройка скриптов и запуск

Чтобы автоматизировать работу, я добавил в `package.json` два скрипта:

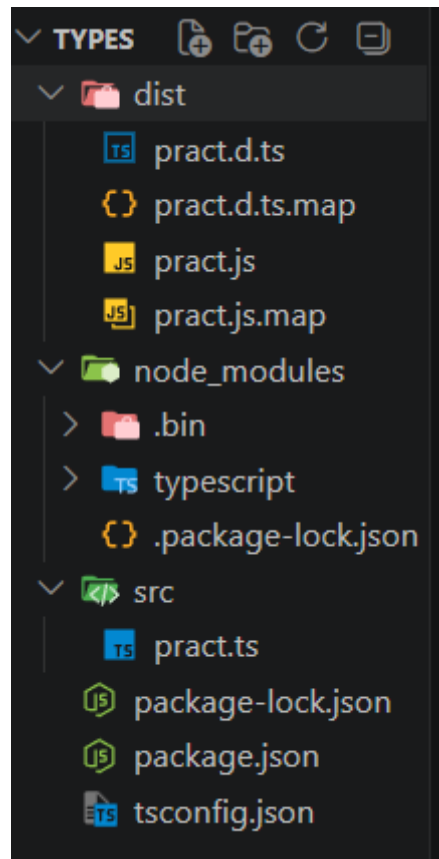
- "build": "tsc -p tsconfig.json" — для компиляции проекта.
- "start": "node dist/pract.js" — для запуска готового кода.

При запуске `npm run build` TypeScript успешно проверил мой код и создал папку `dist` с JS-файлом. После этого я запустил проект командой `npm run start`, и в консоли отобразились все результаты выполнения программы.

Вывод

В ходе работы я на практике увидел преимущество типизации: ошибки (например, неправильный тип данных или отсутствие запятой в JSON) выявляются сразу, а не в процессе работы программы. TypeScript делает код более предсказуемым и понятным для разработки.

Скриншоты процесса работы:



Скриншот 1 - Дерево файлов

```
● PS C:\Users\User\Desktop\WEB\TypeS> npm run start

> types@1.0.0 start
> node dist/pract.js

{
  id: 5,
  company: 'Corporation',
  isPublished: true,
  tags: [ 'TypeScript', 'JavaScript' ],
  x: 10
}
No age of the user
Hello from User.greet()
hello world
ID is equal to ID 123
ID is equal to 123
Contract signed by Anna, email: anna@gmail.com, creditScore: 800
User not authorized
No user was found with that username
Wrong credentials
Internal error
Anna
56
{
  employeeId: 1,
  name: 'Jessica',
  startDate: 2026-05-04T04:57:04.390Z,
  department: 'Finance'
}
```

Скрин 2 - Результат выполнения команды “npm run start”